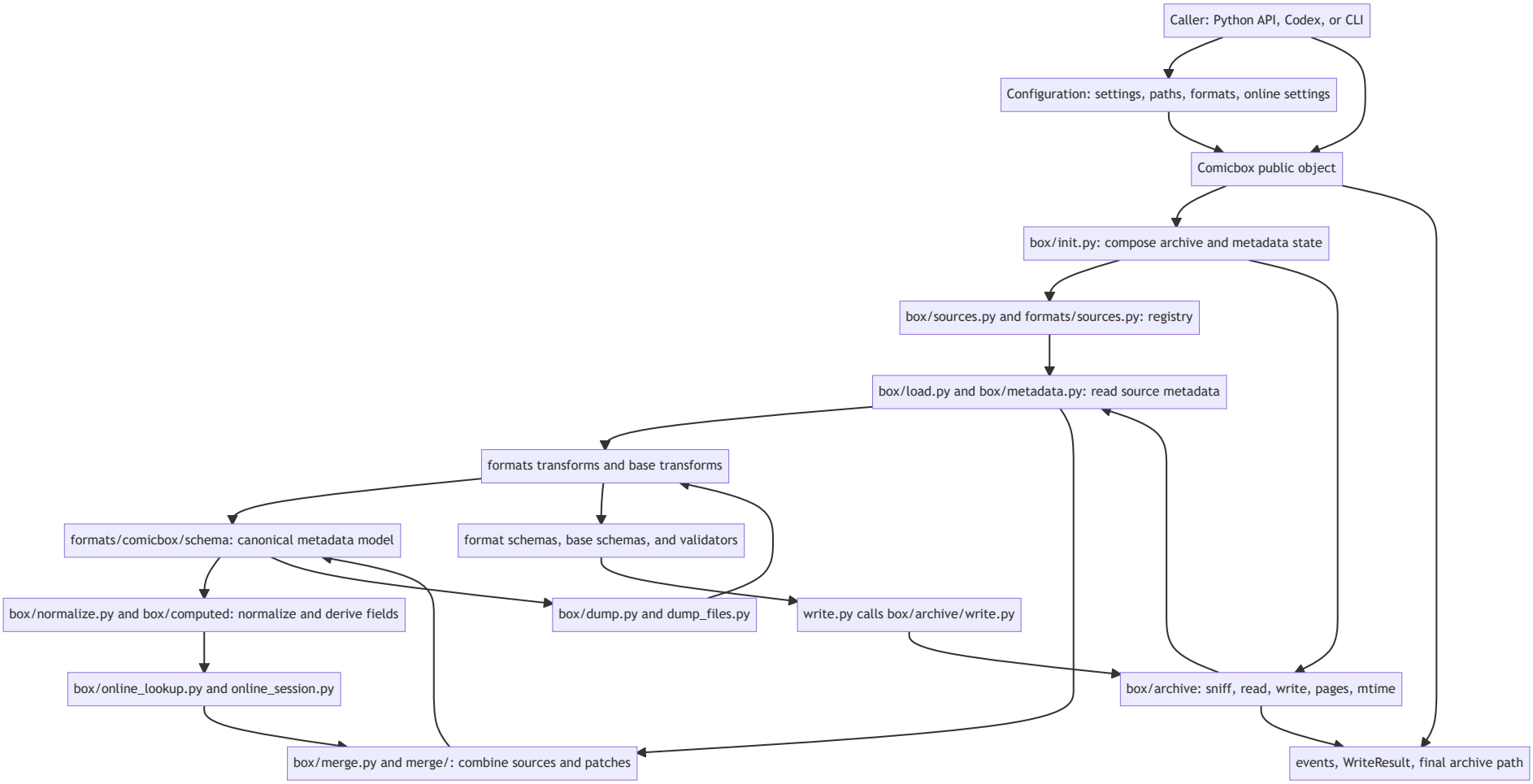
Comicbox Architecture and Workflows

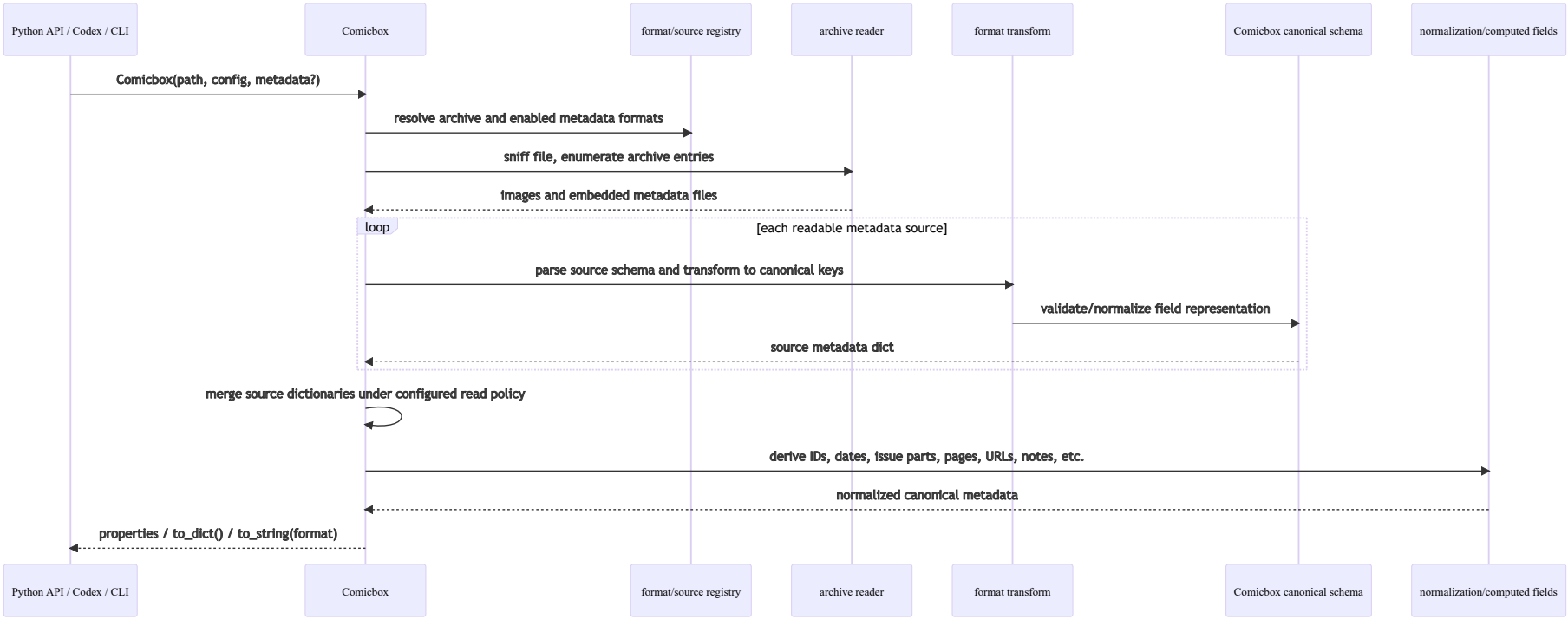
Repository snapshot: local `../comicbox`, `main`, `9cacf3c` (`v5.6.1`, 2026-11-06), clean and tracking `origin/main` when scanned on 2026-11-09. **Purpose:** Module-level map for understanding how the Comicbox API and CLI read, normalize, fetch, merge, transform, validate, and write comic metadata.

These diagrams summarize source organization, not every function call. The repository changes over time; use the paths and commit above when comparing this map with a different checkout.

1. Main architecture

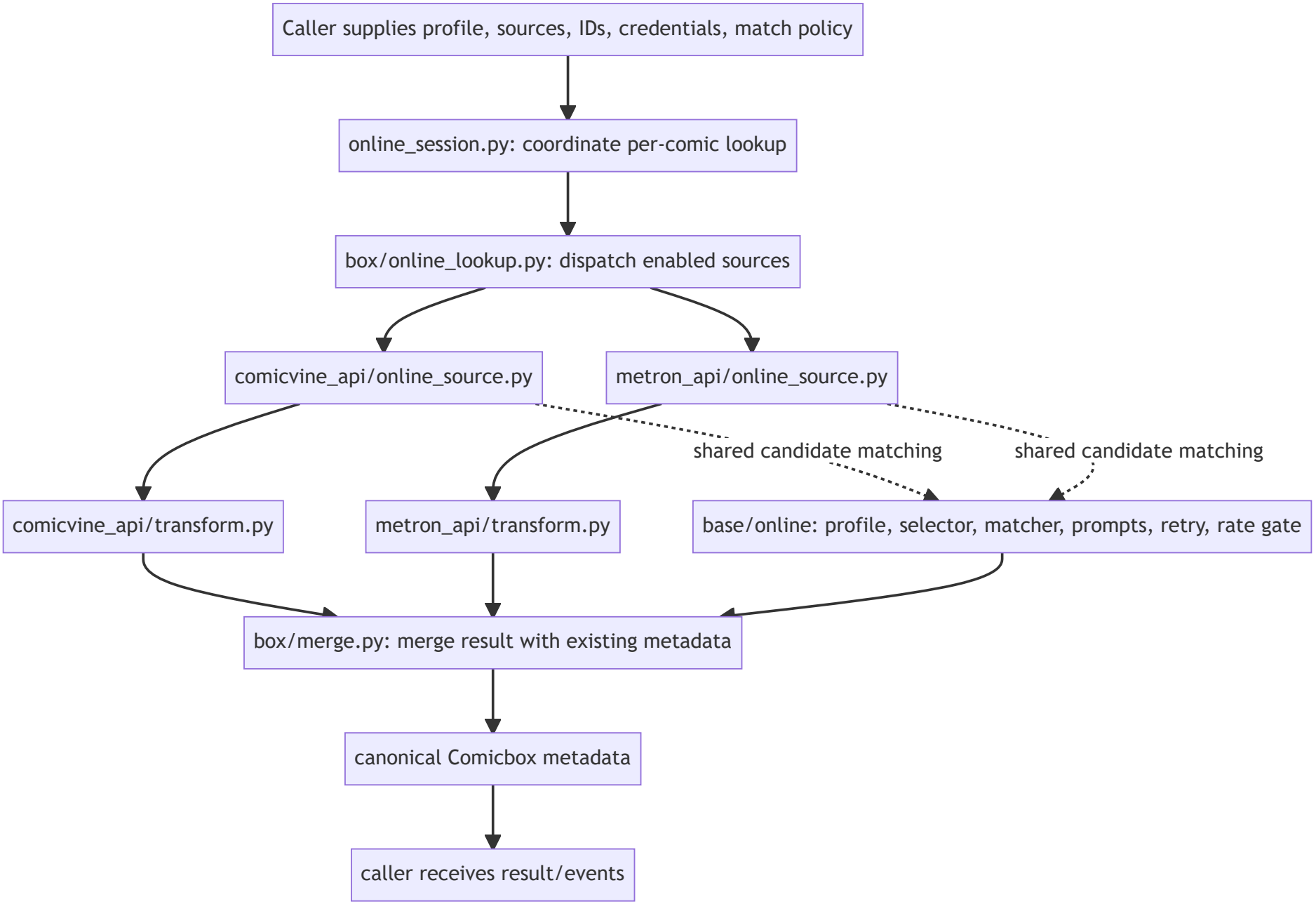

Caller: Python API, Codex, or CLI
Configuration: settings, paths, formats, online settings
Comicbox public object
box/init.py: compose archive and metadata state
box/sources.py and formats/sources.py: registry
box/load.py and box/metadata.py: read source metadata
formats transforms and base transforms
formats/comicbox/schema: canonical metadata model
format schemas, base schemas, and validators
box/normalize.py and box/computed: normalize and derive fields
box/dump.py and dump_files.py
write.py calls box/archive/write.py
box/online_lookup.py and online_session.py
box/archive: sniff, read, write, pages, mtime
box/merge.py and merge/: combine sources and patches
events, WriteResult, final archive path

2. Read/import workflow


Python API / Codex / CLI
Comicbox
format/source registry
archive reader
format transform
Comicbox canonical schema
normalization/computed fields
Comicbox(path, config, metadata?)
resolve archive and enabled metadata formats
sniff file, enumerate archive entries
images and embedded metadata files
loop
[each readable metadata source]
parse source schema and transform to canonical keys
validate/normalize field representation
source metadata dict
merge source dictionaries under configured read policy
derive IDs, dates, issue parts, pages, URLs, notes, etc.
normalized canonical metadata
properties / to_dict() / to_string(format)
Python API / Codex / CLI
Comicbox
format/source registry
archive reader
format transform
Comicbox canonical schema
normalization/computed fields

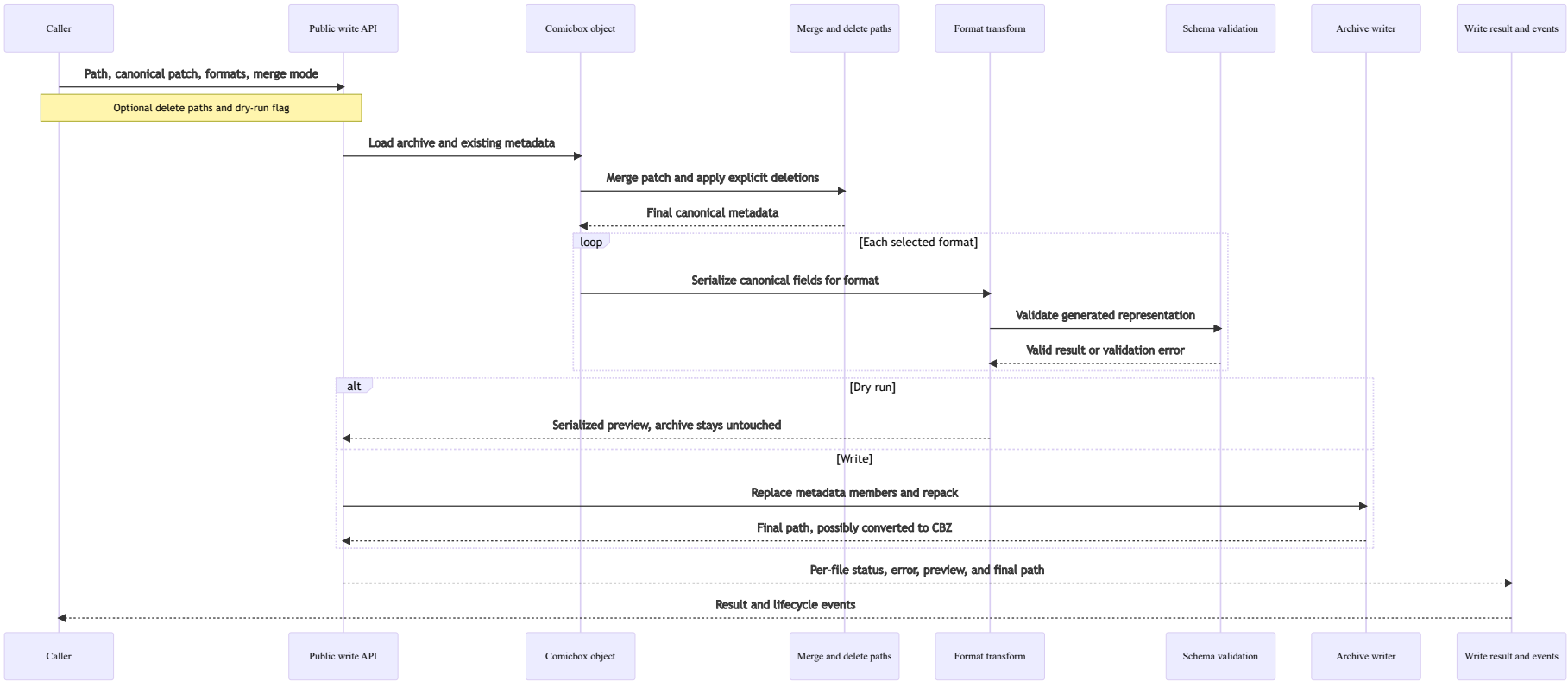
The format registry and transforms separate each container's vocabulary from the canonical Comicbox representation. Relevant format directories include `comic_info`, `metron_info`, `comet`, `comic_book_info`, `pdf`, `filename`, `comicvine_api`, and `metron_api`.

3. Online lookup and enrichment


Caller supplies profile, sources, IDs, credentials, match policy
online_session.py: coordinate per-comic lookup
box/online_lookup.py: dispatch enabled sources
comicvine_api/online_source.py
metron_api/online_source.py
shared candidate matching
shared candidate matching
comicvine_api/transform.py
metron_api/transform.py
base/online: profile, selector, matcher, prompts, retry, rate gate
box/merge.py: merge result with existing metadata
canonical Comicbox metadata
caller receives result/events

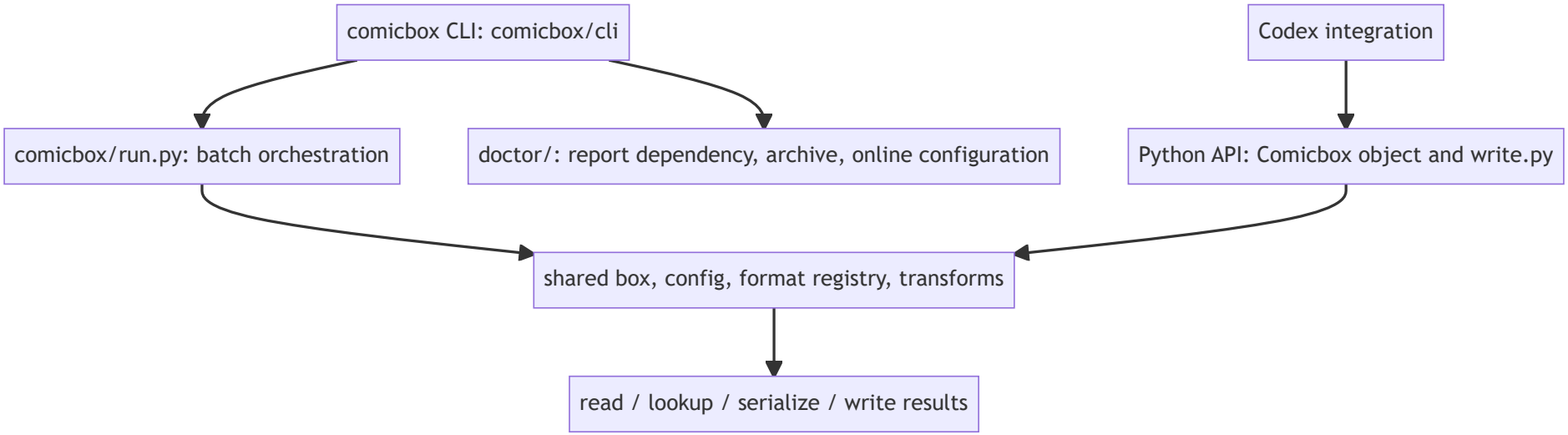
Comic Vine and Metron have source-specific clients and payload transforms; shared online modules handle matching, selection/prompting, credentials, retry, throttling/rate limits, series filtering/cache, and session state. The Metron API transform can preserve a GCD cross-identifier when that identifier is present in Metron's issue payload. The inspected tree has no GCD API source adapter; a local GCD dataset would need its own intake and adapter to canonical metadata.

4. Write workflow


Caller
Public write API
Comicbox object
Merge and delete paths
Format transform
Schema validation
Archive writer
Write result and events
Path, canonical patch, formats, merge mode
Optional delete paths and dry-run flag
Load archive and existing metadata
Merge patch and apply explicit deletions
Final canonical metadata
loop
[Each selected format]
Serialize canonical fields for format
Validate generated representation
Valid result or validation error
alt
[Dry run]
Serialized preview, archive stays untouched
[Write]
Replace metadata members and repack
Final path, possibly converted to CBZ
Per-file status, error, preview, and final path
Result and lifecycle events
Caller
Public write API
Comicbox object
Merge and delete paths
Format transform
Schema validation
Archive writer
Write result and events

Important write concepts: canonical patches; target format selection; `additive`, `replace`, or `update` merge modes; explicit `delete_keys` for clears; dry-run serialization; per-item errors; and a possible final-path change when an archive format is converted to CBZ. Callers that track files must use the result's final path and handle the configured original-retention policy.

5. CLI and public API entry points



Codex is an external caller/orchestrator. Its own task queues, permissions, database bookkeeping, re-import, and UI are not Comicbox modules; Comicbox provides the reusable metadata/archive library underneath those workflows.

6. Directory map by responsibility

Package area	Responsibility
`comicbox/box/`	Public object composition, source loading, metadata normalization, computed fields, merging, dumping, online lookup, archive I/O.
`comicbox/formats/`	Format registry, source adapters, schemas, and transformations between external formats and canonical metadata.
`comicbox/formats/base/`	Shared field/schema/transform machinery and online matching/network policies.
`comicbox/config/`	Typed configuration, defaults, path resolution, online settings, and derived configuration.
`comicbox/merge/`	Merge algorithms and field-level conflict policy.
`comicbox/validate/` and `comicbox/formats/base/schemas/`	JSON/XML/YAML schema validation and error reporting.
`comicbox/cli/` , `comicbox/run.py`	Command-line interface and multi-file orchestration.
`comicbox/doctor/`	Diagnostics for packages, archives, images, settings, and online sources.
`comicbox/identifiers/` , `comicbox/enums/`	Identifier parsing/URLs and normalized vocabularies.
`tests/`	Format, API, online behavior, archive handling, validation, and regression fixtures.

7. Local-GCD integration implications

1. Keep the GCD snapshot importer separate from archive writing; record its provenance, schema/version, date, and applicable data-use terms.
2. Resolve candidate comic matches first, with identifiers and other evidence shown to a reviewer; do not silently equate a title/issue guess with a confirmed record.
3. Convert approved GCD fields into Comicbox's canonical schema, not directly into format-specific XML.
4. Use dry-run output and validate the selected target formats before writes.
5. Preserve existing metadata unless field precedence explicitly authorizes replacement; choose merge mode and deletion paths deliberately.
6. Exercise both XML formats, dual-format archives, malformed/duplicate members, unsupported formats, interrupted writes, and conversion/final-path handling on disposable fixtures before any library batch.

This document is a source-derived architecture map. It does not claim runtime validation, current external API availability, or approval of GCD data use.